

COMPLETE LEARNING GUIDE

Tailwind CSS

Utility-First Styling, Design Systems and Component Thinking

WEEK 03 - TAILWIND CSS & MODERN FRONTEND

Why frameworks exist, how utility-first works, configuring a brand, and building in components.

Prepared for: Get Techy With Lucky / Tech Pulse Insider

Instructor: Lucky Nakola

Learn • Build • Connect • Grow

Course Information

Field	Detail
Program	Web Development Masterclass
Provider	Get Techy With Lucky / Tech Pulse Insider
Instructor	Lucky Nakola
Week	Week 03 of 08
Module	Tailwind CSS + Modern Frontend Development
Estimated study time	6-8 hours
Builds on	Week 02 - CSS Foundations and Responsive Design
Format	Read -> Practise -> Quiz -> Build -> Submit
Assessment	Weekly quiz (auto-scored) and a practical assignment submitted via GitHub

Learning Objectives

By the end of this week you should be able to:

1. Explain why CSS frameworks exist and compare hand-written CSS, Bootstrap and Tailwind honestly.
2. Read and write Tailwind utility classes for spacing, colour, typography, flex and grid.
3. Apply responsive breakpoint prefixes and state variants such as hover, focus and dark.
4. Configure tailwind.config.js so a brand palette and type scale become first-class utilities.
5. Recognise when repeated utility strings should become a reusable component.
6. Set up a real Tailwind build and explain why the CDN script does not belong in production.
7. Rebuild an existing site in Tailwind without losing accessibility or semantics.

Prerequisites

Before starting this week, make sure you can already do the following:

- Write CSS by hand, including selectors, the box model, Flexbox and Grid.
- Explain what a media query does and why mobile-first is the default approach.
- Run commands in a terminal and install packages with npm.
- Write semantic HTML with meaningful elements rather than divs everywhere.

Table of Contents

This guide is written to be read in order. Each chapter builds on the one before it.

1. Introduction - Why CSS Frameworks Exist
2. Utility-First: The Idea Behind Tailwind
3. Setting Tailwind Up Properly
4. The Core Utilities You Will Use Every Day
5. Layout with Tailwind: Flex and Grid Utilities
6. Responsive Design and State Variants
7. Design Systems: Configuring Tailwind for a Brand
8. Component Thinking - Avoiding Copy-Paste Chaos
9. Important Terminology
10. Common Mistakes and How to Avoid Them
11. Professional Considerations: Performance, Accessibility and Team Work
12. Practical Exercise - Rebuild the Week 2 Site in Tailwind
13. Knowledge Check - Weekly Quiz
14. Assignment - The Component-Driven Landing Page
15. Summary

Chapter 1 - Introduction - Why CSS Frameworks Exist

You spent Week 2 writing CSS by hand, and that was the right order. You now know what a framework is doing on your behalf, which means you can judge whether it is helping. Skipping straight to a framework produces developers who can assemble a page but cannot fix one.

Frameworks exist because hand-written CSS on a real project runs into the same four problems every time. Naming gets hard, and teams burn hours arguing about whether it is `.card-title` or `.cardTitle`. Stylesheets only ever grow, because nobody is ever certain a rule is safe to delete. Specificity conflicts creep in until somebody reaches for `!important`. And without a shared scale, spacing drifts: 12px here, 14px there, 15px because it looked right at the time.

WHY THIS MATTERS

A framework is not a shortcut around learning CSS. It is a set of constraints that trade some freedom for consistency and speed. Understanding what you are trading is the difference between choosing a tool and being handed one.

Two Kinds of Framework

	Component frameworks (Bootstrap)	Utility frameworks (Tailwind)
What you get	Finished components: cards, navbars, modals	Single-purpose classes you combine
Speed to first draft	Very fast	Fast, after you learn the names
Looks like	Bootstrap, unless heavily customised	Whatever you design
Customising	Override or fight the framework's CSS	Change the config, or just use different utilities
Markup	Clean, few classes	Noisy, many classes
Best for	Internal tools, admin panels, prototypes	Branded products with their own design

How to Use This Guide

Read this guide with an editor open beside it. Type every example yourself rather than copying and pasting. Typing is slower, and that is the point: it is what turns a concept you have read into a skill you own. When something does not work, resist the urge to look at the answer straight away. Read the error, form a guess about the cause, then test that guess. Debugging is the skill that separates people who can build from people who can only follow.

Chapter 2 - Utility-First: The Idea Behind Tailwind

In traditional CSS you invent a name for a thing, then describe it elsewhere. In Tailwind you describe it in place, using classes that each map to essentially one declaration.

```
<!-- Traditional: a name here, the description somewhere else -->
<div class="card">...</div>

.card {
  background: #fff;
  border-radius: 12px;
  padding: 24px;
  box-shadow: 0 1px 3px rgb(0 0 0 / 0.1);
}

<!-- Tailwind: the description is the class list -->
<div class="bg-white rounded-xl p-6 shadow-sm">...</div>
```

The same card, described two ways.

The honest objection is that the markup gets noisy, and it does. The honest benefit is that you never invent a name, never hunt for which file a style lives in, never worry that deleting a rule breaks a page you forgot about, and never lose a specificity argument, because every utility is a single class of equal weight.

NOTE

Notice what p-6 and rounded-xl really are: they are not arbitrary numbers, they are steps on a scale. Tailwind's real contribution is not short class names. It is that it makes the inconsistent choice slightly harder than the consistent one.

Chapter 3 - Setting Tailwind Up Properly

The Quick Way, and Why It Is Only for Learning

```
<script src="https://cdn.tailwindcss.com"></script>
```

Fine for a lesson or a quick experiment. Never for a site you ship.

The CDN build downloads the entire framework and compiles classes in the browser on every page load. It is slow, it cannot be purged, and it fails without JavaScript. Use it to try an idea in five seconds; use a build step for anything real.

The Real Setup

```
npm init -y
npm install -D tailwindcss postcss autoprefixer
npx tailwindcss init -p
```

Three commands. The -p flag also writes a postcss.config.js for you.

```
// tailwind.config.js
export default {
  content: ['./index.html', './src/**/*.{js,ts,jsx,tsx,html}'],
  theme: {
    extend: {},
  },
}
```

```
},  
plugins: [],  
};
```

The content array is the most important line: it tells Tailwind where to look for class names.

```
/* src/input.css */  
@tailwind base;  
@tailwind components;  
@tailwind utilities;
```

Three directives Tailwind replaces at build time with the generated CSS.

```
npx tailwindcss -i ./src/input.css -o ./dist/output.css --watch
```

Watch mode: rebuilds the stylesheet every time you save a file.

COMMON MISTAKE

If a class you wrote does nothing, check the content array first. Tailwind only generates classes it can literally find as text in the files you listed. This is also why building class names dynamically, like `text-\${color}-500`, silently fails - Tailwind never sees the finished string.

Chapter 4 - The Core Utilities You Will Use Every Day

There are thousands of utilities and you will use perhaps sixty of them constantly. Learn the naming pattern rather than the list: most utilities read as property-then-value, and the numbers come from the spacing scale where one unit is 0.25rem, so p-4 is 1rem.

Purpose	Pattern	Examples
Padding	p / px / py / pt pr pb pl	p-4, px-6, pt-2
Margin	m / mx / my / mt mr mb ml	mt-8, mx-auto, -mt-2
Width and height	w- / h- / min- / max-	w-full, h-screen, max-w-3xl
Text size	text-	text-sm, text-lg, text-3xl
Text colour	text-{colour}-{shade}	text-slate-700, text-brand
Background	bg-{colour}-{shade}	bg-white, bg-blue-600, bg-black/50
Font weight	font-	font-medium, font-bold
Border	border, border-{side}, rounded-	border, border-b, rounded-xl
Shadow	shadow-	shadow-sm, shadow-lg
Display	block / inline / flex / grid / hidden	hidden, md:block
Gap	gap- / gap-x- / gap-y-	gap-6

```
<button class="px-5 py-2.5 rounded-lg bg-blue-700 text-white font-medium  
  hover:bg-blue-800 focus-visible:outline-none  
  focus-visible:ring-2 focus-visible:ring-amber-400  
  transition-colors">
```

```
Get started
</button>
```

A complete, accessible button. Read it left to right: box, colour, type, then states.

Chapter 5 - Layout with Tailwind: Flex and Grid Utilities

Everything you learned in Week 2 applies unchanged. Tailwind simply renames it. If you know the CSS, you already know the utility - display: flex becomes flex, justify-content: space-between becomes justify-between.

```
<!-- The Week 2 site header, in Tailwind -->
<header class="flex items-center justify-between gap-6 px-6 py-4">
  <a href="/" class="font-bold text-lg text-blue-800">Salon Bella</a>
  <nav class="hidden md:flex gap-6">
    <a href="#services" class="hover:text-blue-700">Services</a>
    <a href="#about" class="hover:text-blue-700">About</a>
    <a href="#contact" class="hover:text-blue-700">Contact</a>
  </nav>
</header>
```

hidden md:flex is the whole mobile menu strategy: hide the links on phones, show them from tablet up.

```
<!-- The responsive card grid -->
<div class="grid grid-cols-1 gap-6 md:grid-cols-2 lg:grid-cols-3">
  <article class="rounded-xl border border-slate-200 bg-white p-6 shadow-sm">...</article>
  <article class="rounded-xl border border-slate-200 bg-white p-6 shadow-sm">...</article>
  <article class="rounded-xl border border-slate-200 bg-white p-6 shadow-sm">...</article>
</div>
```

One column, then two, then three. Notice the repetition in the cards - Chapter 8 deals with that.

Chapter 6 - Responsive Design and State Variants

A variant is a prefix that limits when a utility applies. Everything unprefixed applies always; a prefix narrows it to a screen size or a state. Because breakpoints are min-width, Tailwind is mobile-first whether you think about it or not.

Prefix	Applies from	Typical use
(none)	All screens	The phone layout
sm:	640px	Large phones
md:	768px	Tablets
lg:	1024px	Laptops
xl:	1280px	Large desktops
2xl:	1536px	Very large displays

```
<h1 class="text-2xl md:text-4xl lg:text-5xl font-bold">
  Beautiful hair, every time
</h1>
```

One heading, three sizes. Read it as: 2xl normally, bigger from tablet, bigger again from laptop.

State and Preference Variants

Variant	Fires when	Example
hover:	Pointer is over the element	hover:bg-blue-800
focus-visible:	Focused via keyboard	focus-visible:ring-2
active:	Being pressed	active:scale-95
disabled:	The control is disabled	disabled:opacity-50
group-hover:	An ancestor marked group is hovered	group-hover:translate-x-1
dark:	Dark mode is active	dark:bg-slate-900
motion-reduce:	User asked for less motion	motion-reduce:transition-none

BEST PRACTICE

Prefer focus-visible: over focus:. Plain focus: also fires on mouse clicks, which puts a ring around buttons people just clicked. focus-visible: shows the ring when it actually helps - for keyboard users navigating with Tab.

Chapter 7 - Design Systems: Configuring Tailwind for a Brand

Out of the box every Tailwind site looks like every other Tailwind site, because everyone uses the same default palette. The config file is where you stop that. Extend the theme with your brand and the utilities generate themselves.

```
// tailwind.config.js
export default {
  content: ['./index.html', './src/**/*.{js,html}'],
  darkMode: 'class',
  theme: {
    extend: {
      colors: {
        brand: {
          DEFAULT: '#003f9e',
          light: '#005ce6',
          dark: '#002a6b',
        },
      },
      accent: '#ffcc00',
    },
  },
  fontFamily: {
```



```
sans: ['Inter', 'system-ui', 'sans-serif'],
},
borderRadius: {
  xl: '12px',
},
},
},
},
};
```

Extending the theme adds *bg-brand*, *text-brand-light*, *ring-accent* and so on.

WHY THIS MATTERS

Use `extend` rather than replacing theme wholesale. Replacing deletes Tailwind's entire default scale, and you will spend the rest of the project rebuilding greys you did not mean to lose.

Once the brand lives in the config, arbitrary values like `bg-[#003f9e]` become a smell. If you find yourself writing one, that usually means the value belongs in the config instead.

Chapter 8 - Component Thinking - Avoiding Copy-Paste Chaos

The card grid in Chapter 5 repeated the same seven classes three times. With six cards it would be six copies, and when the design changes you would need to find every one. This is the moment to extract.

Option One: A Component

If you are working in a component framework - React, Vue, or even PHP includes later in this course - extract the markup into a component and pass the content in. This is the preferred route because it keeps both the markup and the styling in one place.

```
// Card.jsx
export function Card({ title, children }) {
  return (
    <article className="rounded-xl border border-slate-200 bg-white p-6 shadow-sm
      dark:border-slate-700 dark:bg-slate-800">
      <h3 className="font-semibold text-lg">{title}</h3>
      <div className="mt-2 text-slate-600 dark:text-slate-300">{children}</div>
    </article>
  );
}
```

One definition, used everywhere. Change the design once.

Option Two: `@apply`

In a plain HTML project without components, `@apply` lets you fold a utility string into a single class. Use it sparingly - overusing it recreates the stylesheet Tailwind was meant to replace.

```
@layer components {
```

```
.card {  
  @apply rounded-xl border border-slate-200 bg-white p-6 shadow-sm;  
}  
  
.btn-primary {  
  @apply px-5 py-2.5 rounded-lg bg-brand text-white font-medium  
    hover:bg-brand-light focus-visible:ring-2 focus-visible:ring-accent;  
}
```

Reserve `@apply` for genuinely repeated, stable patterns such as buttons and cards.

COMMON MISTAKE

Do not start a project by `@apply`-ing everything into semantic class names. That gives you all of Tailwind's setup cost and none of its benefits. Write utilities first, and extract only once a pattern has actually repeated three or more times.

Chapter 9 - Important Terminology

Most of these are Tailwind's vocabulary for ideas you already met in Week 2. Where a term has a plain-CSS equivalent, make sure you can name both.

Term	Definition	In Plain Words	Example / Use Case
CSS framework	A prewritten CSS library that gives you a starting set of styles or tools.	Someone else's CSS you build on top of.	Tailwind CSS, Bootstrap, Bulma
Utility-first	A design approach where styles are applied as many single-purpose classes in the markup.	Small classes that each do one thing.	<code>p-4 text-lg font-bold</code>
Utility class	A class that sets one specific CSS declaration.	One class, one job.	<code>mt-4</code> sets margin-top.
Component library	A framework that ships prebuilt components such as cards and navbars.	Ready-made UI pieces.	Bootstrap's <code>.card</code>
Tailwind CSS	A utility-first CSS framework that generates classes from a configurable design system.	A toolkit of tiny classes plus a brand config.	<code>class="px-4 py-2 rounded-lg"</code>
Design system	The agreed set of colours, spacing, type sizes and radii a product uses.	The brand's rulebook for the interface.	A defined spacing scale of 4, 8, 16, 24px
Design token	A single named value in a design system.	One named setting.	<code>brand-600</code> , <code>space-4</code>
Spacing scale	The fixed ladder of spacing values a design system allows.	The only gaps you are allowed to use.	<code>p-1 p-2 p-4 p-8</code>
Arbitrary value	A one-off value written inline in square brackets when the scale does not cover it.	An escape hatch for odd values.	<code>w-[347px]</code>
Variant	A prefix that applies a utility only in a certain state or at a certain	'Only when...' in front of a class.	<code>hover:bg-blue-700</code> , <code>md:flex</code>

Term	Definition	In Plain Words	Example / Use Case
	size.		
Breakpoint prefix	A responsive variant applying a utility from a screen width upward.	Turn this on from tablet up.	md:grid-cols-2
Mobile-first (Tailwind)	Unprefixed utilities apply everywhere; prefixed ones apply upward from that width.	Base is the phone, prefixes add for bigger screens.	grid-cols-1 md:grid-cols-3
State variant	A variant keyed to an interaction or element state.	Style for hover, focus, disabled.	focus-visible:ring-2
Dark mode variant	A variant applying utilities when dark mode is active.	Styles for dark theme.	dark:bg-slate-900
Group	A parent marked so children can react to the parent's state.	Hover the card, change the arrow.	group and group-hover:translate-x-1
Purge / content scanning	Tailwind reads your files and only generates the classes it actually finds.	Unused CSS never gets built.	The content array in the config
JIT engine	Tailwind's just-in-time compiler that generates classes on demand as you type.	Builds only what you use, instantly.	Enabled by default in Tailwind 3
tailwind.config.js	The file where you extend or override Tailwind's defaults.	Where your brand is defined.	theme.extend.colors
theme.extend	The config section that adds to the defaults instead of replacing them.	Add your colours, keep Tailwind's.	extend: { colors: { brand: '#003f9e' } }
@apply	A directive that pulls utility styles into a normal CSS class.	Bundle utilities into one class name.	.btn { @apply px-4 py-2 rounded; }
Directive	A special at-rule Tailwind processes at build time.	An instruction to the Tailwind compiler.	@tailwind, @apply, @layer
@layer	A directive placing custom CSS into base, components or utilities.	Tells Tailwind where your CSS belongs.	@layer components { }
Preflight	Tailwind's built-in CSS reset.	Clears browser defaults before you start.	Removes default heading sizes and margins
PostCSS	A tool that transforms CSS with plugins; Tailwind is one such plugin.	The build step CSS passes through.	postcss.config.js
Container	A utility that sets max-width to the current breakpoint.	A centred content column.	class="container mx-auto"
Flexbox utilities	Tailwind classes that map onto flexbox properties.	Flexbox, but as class names.	flex items-center justify-between
Grid utilities	Tailwind classes that map onto CSS Grid properties.	Grid, but as class names.	grid grid-cols-3 gap-6
Ring	A Tailwind outline drawn with box-shadow, used mostly for focus states.	A visible halo for focus.	focus:ring-2 ring-blue-500
Opacity modifier	A slash suffix that applies transparency to a colour utility.	Same colour, see-through.	bg-black/50
Component	A reusable, self-contained piece of interface.	One UI building block used many times.	A card, a button, a navbar

Term	Definition	In Plain Words	Example / Use Case
Composition	Building larger interfaces by combining smaller components.	Small pieces make big pages.	A page made of header, hero and card grid
Separation of concerns	Keeping unrelated responsibilities in different places.	Each part does one job.	Markup, styling and behaviour kept distinct
Specificity (in Tailwind)	Utilities are all single classes, so they carry equal, low specificity.	No specificity wars.	Order in the generated sheet decides, not selector weight
Class soup	Markup made unreadable by very long lists of utility classes.	Too many classes in one tag.	The signal to extract a component
Build step	The compilation that turns source files into what the browser loads.	Turning your code into shippable files.	npm run build
CDN build	Loading Tailwind from a script tag rather than compiling it.	Quick Tailwind, no setup.	Fine for a demo, never for production
Bundle size	How many kilobytes the browser must download.	How heavy your site is.	A purged Tailwind build is often under 15KB
Prototype	A quick, rough build made to test an idea.	A draft you expect to throw away.	A landing page mocked in an afternoon
Consistency	Using the same values and patterns throughout an interface.	Everything feels like one product.	One spacing scale used everywhere
Accessibility (a11y)	Designing so people with disabilities can use your product.	Everyone can actually use it.	Visible focus rings, real labels, good contrast
prefers-reduced-motion	A user setting asking for less animation, exposed to CSS.	Some people get sick from motion.	motion-reduce:transition-none
Semantic HTML (still)	Using meaningful elements regardless of how you style them.	A button is still a <button>.	Tailwind never replaces good markup

Chapter 10 - Common Mistakes and How to Avoid Them

Mistake	What goes wrong	The fix
Shipping the CDN build	Huge download, browser-side compilation, no purge	Install via npm and run a real build
Building class names dynamically	Tailwind never sees the string, so no CSS is generated	Write complete class names, or safelist them
Arbitrary values everywhere	The design scale is bypassed and consistency dies	Put the value in theme.extend and use a named utility
Replacing theme instead of extending	Tailwind's default palette and scale vanish	Always nest your additions under extend
Never extracting components	The same 30 classes copied into eight places	Extract to a component or an @apply class after the third repeat
@apply for everything	You have rebuilt a normal stylesheet, with extra build steps	Utilities first; @apply only for stable, repeated patterns
Using divs for buttons	No keyboard access, no semantics,	Use <button>; style it with utilities

Mistake	What goes wrong	The fix
	no default behaviour	
focus: instead of focus-visible:	Rings appear after mouse clicks and look like bugs	Use focus-visible: for keyboard focus
Forgetting dark mode contrast	Text that passes on white fails on slate-900	Check contrast in both modes
Missing the content path	Classes in a new folder silently do nothing	Keep the content array up to date as the project grows

Chapter 11 - Professional Considerations: Performance, Accessibility and Team Work

Performance

A purged Tailwind build for a medium site is usually under 15KB gzipped, which is smaller than most hand-written stylesheets, because unused rules are never generated. That advantage disappears entirely if you ship the CDN build, so the build step is a performance decision, not a formality.

Accessibility Is Still Yours

- **Tailwind has no opinion about semantics.** It will style a div as convincingly as a button, and a screen reader will not be fooled.
- **Preflight removes default heading sizes.** An h1 will not look like a heading until you style it - do not respond by using a div.
- **Colour utilities do not check contrast.** text-slate-400 on bg-white fails. Verify it.
- **Keep a visible focus state.** Preflight strips default outlines; put a ring back.

Working in a Team

Utility classes make code review easier in one specific way: a reviewer can see exactly what a change does without opening another file. They make it harder in another: long class strings are hard to scan. Teams handle this by ordering utilities consistently - layout, then box, then typography, then colour, then state - and most use the official Prettier plugin to sort them automatically.

Chapter 12 - Practical Exercise - Rebuild the Week 2 Site in Tailwind

8. Create a new folder and run the three setup commands. Confirm the build produces an output.css.
9. Link the compiled stylesheet and verify Tailwind is live by adding bg-slate-100 to the body.
10. Copy your Week 2 HTML across, stripping every class and the old stylesheet link.
11. Add your brand colours, font and radius to theme.extend in the config.
12. Rebuild the header with flex, items-center and justify-between.
13. Hide the nav links below md and show them from md up. Add a hamburger button for small screens.
14. Rebuild the hero: spacing, a responsive heading size, and two buttons of differing prominence.
15. Rebuild the card grid with grid-cols-1 md:grid-cols-2 lg:grid-cols-3 and a gap.
16. Give each card the same utility string, then extract it once the third copy exists.
17. Rebuild the contact form with consistent field styling and a focus-visible ring.
18. Enable darkMode: 'class' and add dark: variants across every section.
19. Add a toggle button that flips the dark class on the html element.
20. Check every breakpoint and both colour modes. Fix contrast failures.
21. Run the production build and note the size of the output CSS. Compare it with your Week 2 stylesheet.

BEST PRACTICE

Step 14 is the one worth writing down. Seeing your Tailwind bundle come out smaller than the CSS you hand-wrote is what makes the purge step click.

Chapter 13 - Knowledge Check - Weekly Quiz

These questions test whether you understand what Tailwind is doing, not whether you have memorised class names. Class names you can look up; the model you cannot.

Pass mark: 70% | *Answers are at the end of this chapter.*

Q1. What does 'utility-first' mean in Tailwind?

- a) Tailwind ships finished components you drop in
- b) You compose designs from many small single-purpose classes in the markup
- c) Utilities are loaded before your own CSS
- d) You must write all CSS in one utility file

Q2. In Tailwind, what does the class md:grid-cols-3 do?

- a) Three columns only on medium screens
- b) Three columns from the medium breakpoint upward
- c) Three columns below the medium breakpoint
- d) Three columns on every screen

Q3. Why is a production Tailwind build usually small despite the framework having thousands of utilities?

- a) The classes are compressed at runtime
- b) Tailwind scans your files and only generates classes it finds
- c) Browsers cache the framework
- d) Unused classes are downloaded lazily

Q4. Where should you define your brand colours so they become Tailwind utilities?

- a) Inline with arbitrary values everywhere
- b) In `theme.extend.colors` in `tailwind.config.js`
- c) In a separate `brand.css` file
- d) In the HTML style attribute

Q5. Using Tailwind means you no longer need to understand CSS.

True / False

Q6. The Tailwind CDN script is a reasonable choice for a production website.

True / False

Q7. Wrapping a repeated block of utility classes into a component is a sign you are using Tailwind wrongly.

True / False

Q8. Explain the trade-off between writing traditional CSS and writing Tailwind utilities. Give one honest advantage of each.

Short answer - write two or three sentences.

Q9. Your card markup has grown to 30 utility classes and appears in six places. What should you do and why?

Short answer - write two or three sentences.

Q10. Name two accessibility responsibilities Tailwind does not handle for you.

Short answer - write two or three sentences.

Answer Key

Mark your own work honestly - the goal is to find your gaps, not to score well.

#	Answer	Why
Q1	b - You compose designs from many small single-purpose classes	Each utility sets essentially one declaration. The design emerges from combining them, rather than from a prewritten component.
Q2	b - Three columns from the medium breakpoint upward	Tailwind variants are min-width based. A prefix applies at that width and everything larger, which is why Tailwind is mobile-first by default.
Q3	b - Tailwind scans your files and only generates	The content array in the config tells Tailwind where to look. Anything it never sees is never generated, so the shipped CSS holds only what you

#	Answer	Why
	classes it finds	used.
Q4	b - In <code>theme.extend.colors</code> in <code>tailwind.config.js</code>	Extending the theme makes <code>bg-brand</code> , <code>text-brand</code> and <code>border-brand</code> available everywhere, and keeps one source of truth for the palette.
Q5	False	Every utility is a thin wrapper over a CSS declaration. Debugging a Tailwind layout is debugging CSS - you simply read the class names instead of a stylesheet.
Q6	False	The CDN build ships the whole framework and compiles in the browser. It is convenient for a demo or a lesson, but it is slow and unpurged, so production needs a real build step.
Q7	False	It is the opposite - extraction is the intended workflow. When a pattern repeats, you turn it into a component or an <code>@apply</code> class so there is one place to change it.
Q8	Traditional CSS keeps markup clean and expresses intent through semantic class names, but tends toward growing stylesheets, naming debates and specificity conflicts. Tailwind keeps styling next to the markup, enforces a design scale and ships almost no unused CSS, but makes markup noisier and requires a build step.	Being able to argue both sides is what makes the choice a decision rather than a habit.
Q9	Extract it into a reusable component, or an <code>@apply</code> -based class if there is no component layer. It gives one place to change the design and removes the risk of the six copies drifting apart.	Repetition is the trigger for extraction. Duplicated utility strings are the Tailwind equivalent of copy-pasted CSS.
Q10	Using semantic elements (a real <code><button></code> , real labels associated with inputs) and ensuring colour combinations meet contrast requirements. Tailwind will happily generate <code>text-yellow-200</code> on <code>bg-white</code> .	A framework produces styles, not judgement. Accessibility stays the developer's responsibility.

Chapter 14 - Assignment - The Component-Driven Landing Page

Objective

Rebuild your Week 2 responsive business site using Tailwind CSS, configured with a real brand theme, and refactored into reusable components rather than repeated class strings.

Scenario

The business from Week 2 now wants a proper marketing landing page: a hero, a features section, a pricing or services comparison, testimonials, and a call to action. They also want it to look right in dark mode, because half their customers browse at night.

Requirements

- A responsive navigation bar with a working mobile menu.
- A hero section with heading, supporting copy, and two call-to-action buttons of differing prominence.
- A features section of at least three cards using Tailwind grid utilities.
- A services or pricing section presenting at least three tiers side by side on desktop and stacked on mobile.
- A testimonials section with at least two quotes.
- A footer with navigation, contact details and social links.
- A working dark mode across every section.
- At least three repeated patterns extracted into reusable components or @apply classes.

Expected Output

A Tailwind project with a real build step (not the CDN script), a configured tailwind.config.js carrying the brand palette and type scale, and a compiled stylesheet. The page must work from 320px to 1440px and in both light and dark mode.

Technical Requirements

- Tailwind installed via npm with a build step - the CDN script will not be accepted.
- Brand colours, fonts and any custom spacing declared in theme.extend, not as arbitrary values scattered through the markup.
- Mobile-first: unprefixed utilities for the phone, breakpoint prefixes upward.
- Dark mode implemented with the dark: variant and a working toggle or the system preference.
- Visible focus states on every interactive element, using focus-visible and a ring utility.
- Semantic HTML throughout - nav, main, section, footer, and real button and label elements.
- No more than two arbitrary values in the whole project, each justified in the README.

- The production build committed or reproducible with a documented npm command.

Submission Instructions

22. Create a public GitHub repository named week-03-tailwind-landing.
23. Include package.json, tailwind.config.js and your source files. Do not commit node_modules.
24. Write a README.md covering: how to install and build, your colour and type decisions, which patterns you extracted into components and why.
25. Include light-mode and dark-mode screenshots in the README.
26. Submit the repository link on the Week 3 assignment page in the student portal.

Evaluation Criteria

Criterion	What Earns Full Marks	Weight
Tailwind fluency	Utilities used idiomatically; no fighting the framework with custom CSS	25%
Design system config	Brand tokens defined in the config and used consistently	20%
Component extraction	Repeated patterns extracted; markup stays readable	20%
Responsiveness and dark mode	Works across all widths and both colour schemes	20%
Accessibility	Semantic elements, visible focus, contrast passes in both modes	10%
Documentation	README explains the build and the design decisions	5%

Bonus Challenges

- Add a scroll-triggered reveal animation that respects prefers-reduced-motion.
- Build a reusable Tailwind plugin for one custom utility your design needs.
- Add a working contact form with client-side validation and clear error states.
- Deploy to Vercel or Netlify and put the live link at the top of the README.

Chapter 15 - Summary

- Frameworks exist to solve naming, growth, specificity and inconsistency - not to save you from learning CSS.
- Utility-first means composing from small single-purpose classes instead of inventing component names.
- Every Tailwind utility is a thin wrapper over a CSS declaration you already know from Week 2.
- Variants narrow when a utility applies: breakpoint prefixes are min-width, so Tailwind is mobile-first by default.
- The config file is where a generic Tailwind site becomes your brand. Extend the theme; do not replace it.

- Tailwind only generates classes it can literally find in your content paths - dynamic class names silently fail.
- When a utility string repeats three times, extract it into a component or an `@apply` class.
- Semantics, contrast and focus states remain your responsibility. No framework supplies judgement.

You are not expected to remember everything in this guide. Professional developers look things up constantly. What you are expected to build is a mental model: knowing that a thing exists, roughly how it behaves, and where to look it up when you need the detail.

Further Reading and References

Documentation changes faster than any printed guide. Learning to read the official docs is part of the skill you are building.

- **Tailwind CSS documentation** - <https://tailwindcss.com/docs>
- **Tailwind: Utility-First Fundamentals** - <https://tailwindcss.com/docs/utility-first>
- **Tailwind: Responsive Design** - <https://tailwindcss.com/docs/responsive-design>
- **Tailwind: Theme Configuration** - <https://tailwindcss.com/docs/theme>
- **Tailwind: Dark Mode** - <https://tailwindcss.com/docs/dark-mode>
- **Prettier plugin for Tailwind class sorting** - <https://github.com/tailwindlabs/prettier-plugin-tailwindcss>
- **MDN CSS reference - still the source of truth underneath** - <https://developer.mozilla.org/en-US/docs/Web/CSS>